



Chapter 2

Complexity of Algorithms

Data Structures and Algorithms

Dr. Nguyen Ho Man Rang
Faculty of Computer Science and Engineering
University of Technology, VNU-HCM

- **L.O.1.1** - Define concept “computational complexity” and its special cases, best, average, and worst.
- **L.O.1.2** - Analyze algorithms and use Big-O notation to characterize the computational complexity of algorithms composed by using the following control structures: sequence, branching, and iteration (not recursion).
- **L.O.1.3** - List, give examples, and compare complexity classes, for examples, constant, linear, etc.
- **L.O.1.4** - Be aware of the trade-off between space and time in solutions.
- **L.O.1.5** - Describe strategies in algorithm design and problem solving.



Contents

① Algorithm Efficiency

② Asymptotic Analysis

③ Problems and common complexities

④ P and NP Problems





This materials reused some slides from the following source:

- course CS2210 Data Structures and Algorithms instructed by Professor Olga Veksler Department of Computer Science, University of Western Ontario.

Algorithm
Efficiency

Asymptotic
Analysis

Problems and
common
complexities

P and NP
Problems



Algorithm Efficiency

Compare Algorithms

- Given 2 or more algorithms to solve the same problem, how do we select the best one?
- Some criteria for selecting an algorithm
 - Is it easy to implement, understand, modify?
 - How long does it take to run it to completion?
 - How much of computer memory does it use?
- Software engineering is primarily concerned with the first criteria
- In this course we are interested in the second and third criteria





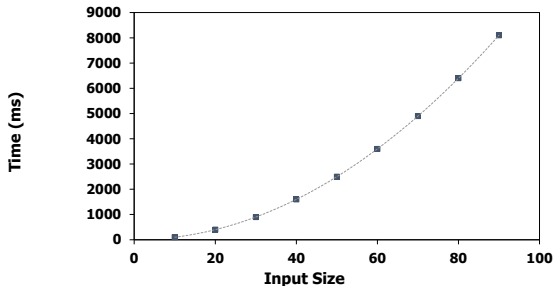
- **Time complexity:** the amount of time that an algorithm needs to run to completion.
- **Space complexity:** the amount of memory an algorithm needs to run.
- We will occasionally look at space complexity, but we are mostly interested in time complexity in this course.
- Thus in this course the better algorithm is the one which runs faster (has smaller time complexity).

How to Calculate Running time

- Most algorithms transform input objects into output objects.



- The running time of an algorithm typically grows with the input size $f(n)$.



How to Calculate Running Time

- Even on inputs of the same size, running time can be very different.
 - Example: algorithm that searches an array containing n integers to find the one with a particular value K (assume that K appears exactly once in the array)
- Idea: analyze running time in the
 - best case
 - worst case
 - average case



How to Calculate Running Time

- Best case running time is usually useless
- Average case time is very useful but often difficult to determine
- We focus on the worst case running time
 - Easier to analyze
 - Crucial to applications such as games, finance and robotics



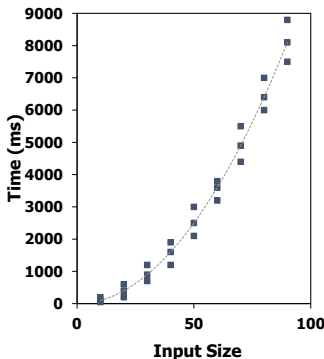


- There are two ways to compare running time between algorithms:
 - Experimental evaluation
 - Theoretical evaluation

Experimental Evaluation of Running Time



- Write a program implementing the algorithm
- Run the program with inputs of varying size and composition
- Measure accurately the actual running time
- Plot the results



Limitations of Experiments

- It is necessary to implement the algorithm, which may be difficult
- Results may not be indicative of the running time on other inputs not included in the experiment
- In order to compare two algorithms, the same hardware and software environments must be used



Theoretical Analysis of Running Time

- Uses a pseudo-code description of the algorithm instead of an implementation
- Characterizes running time as a function of the input size, **n**
- Takes into account all possible inputs
- Allows us to evaluate the speed of an algorithm independent of the hardware/software environment





- For theoretical analysis, we will count **primitive** or **basic** operations, which are simple computations performed by an algorithm
- Example:
 - Evaluating an expression: $x^2 + 3x$
 - Assigning a value to a variable: $x = y$
 - Indexing into an array: $a[10]$
 - Calling a function: $mySwap(a, b)$
 - Returning from a function: $return(x)$

Counting Primitive Operations



- By inspecting the pseudocode, we can determine the maximum number of primitive operations executed by an algorithm, as a function of the input size

Algorithm findMax(a, n)

```
currentMax = a[0]           //2
i = 1                       //2
while (i < n)                 //n
    if (currentMax < a[i])    //2n-2
        currentMax = a[i]    //2n-2
    i = i + 1                 //2n-2
return currentMax            //1
```




- Algorithm `findMax()` executes $7n - 1$ primitive operations in the worst case. Define:
 - a = Time taken by the fastest primitive operation
 - b = Time taken by the slowest primitive operation
- Let $f(n)$ be worst-case time of `arrayMax`.
Then $a(7n - 1) \leq f(n) \leq b(7n - 1)$
- Hence, the running time $f(n)$ is bounded by two linear functions



- Changing the hardware/software environment
 - Affects $f(n)$ by a constant factor, but
 - Does not alter the growth rate of $f(n)$
- Thus we focus on the big-picture which is the growth rate of an algorithm
- The linear growth rate of the running time $f(n)$ is an intrinsic property of algorithm `findMax()`

Linear Loops

```
for (i = 0; i < n; i++)  
    application code
```

The number of times the body of the loop is replicated is n .

$$f(n) = n$$

```
for (i = 0; i < n; i += 2)  
    application code
```

The number of times the body of the loop is replicated is $n/2$.

$$f(n) = n/2$$



Logarithmic Loops

Multiply loops

```
i = 1
while (i <= n)
    application code
    i = i x 2
```

Divide loops

```
i = n
while (i >= 1)
    application code
    i = i / 2
```

The number of times the body of the loop is replicated is

$$f(n) = \log_2 n$$



Nested Loops



Iterations = Outer loop iterations $\times$ Inner loop iterations

Example

```
i = 1
while (i <= n)
    j = 1
    while (j <= n)
        application code
        j = j * 2
    i = i + 1
```

The number of times the body of the loop is replicated is

$$f(n) = n \log_2 n$$

Quadratic Loops



Example

```
i = 1
while (i <= n)
    j = 1
    while (j <= n)
        application code
        j = j + 1
    i = i + 1
```

The number of times the body of the loop is replicated is

$$f(n) = n^2$$

Dependent Quadratic Loops



Example

```
i = 1
while (i <= n)
    j = 1
    while (j <= i)
        application code
        j = j + 1
    i = i + 1
```

The number of times the body of the loop is replicated is

$$1 + 2 + \dots + n = n(n + 1)/2$$

- The growth rate is not affected by
 - constant factors or
 - lower-order terms
- Examples
 - $10^3n + 10^5$ is a linear function
 - $10n^2 + 10^4n$ is a quadratic function
- How do we get rid of the constant factors to focus on the essential part of the running time?





- Algorithm efficiency is considered with only **big problem sizes**.
- We are **not concerned** with an **exact measurement** of an algorithm's efficiency.
- Terms that do **not substantially change** the function's magnitude are eliminated.



Asymptotic Analysis



There are three common notations for asymptotic analysis:

- Big-Oh: $O(.)$
- Big-Omega: $\Omega(.)$
- Big-Theta: $\Theta(.)$

Big-Oh notation

- The big-Oh notation is used widely to characterize running times and space bounds
- The big-Oh notation allows us to ignore constant factors and lower order terms and focus on the main components of a function which affect its growth





- Given functions $f(n)$ and $g(n)$, we say that $f(n)$ is $O(g(n))$ if there are positive constants c and n_0 such that:
$$f(n) \leq c.g(n) \text{ for } n \geq n_0$$
- Example: $2n + 10$ is $O(n)$
- Why?

Big-Oh Rules

Example

$$f(n) = c.n \Rightarrow f(n) = O(n)$$

$$f(n) = n(n+1)/2 = n^2/2 + n/2 \Rightarrow f(n) = O(n^2)$$

- Set the **coefficient** of the term to one.
- Keep the **largest term** and discard the others.

Some example of Big-O:

$$1 \quad \log_2 n \quad n \quad n \log_2 n \quad n^2 \quad \dots \quad n^k \quad \dots \quad 2^n \quad n!$$



- The big-Oh notation gives an upper bound on the growth rate of a function
- The statement “ $f(n)$ is $O(g(n))$ ” means that the growth rate of $f(n)$ is no more than the growth rate of $g(n)$
- We can use the big-Oh notation to rank functions according to their growth rate



Standard Measures of Efficiency

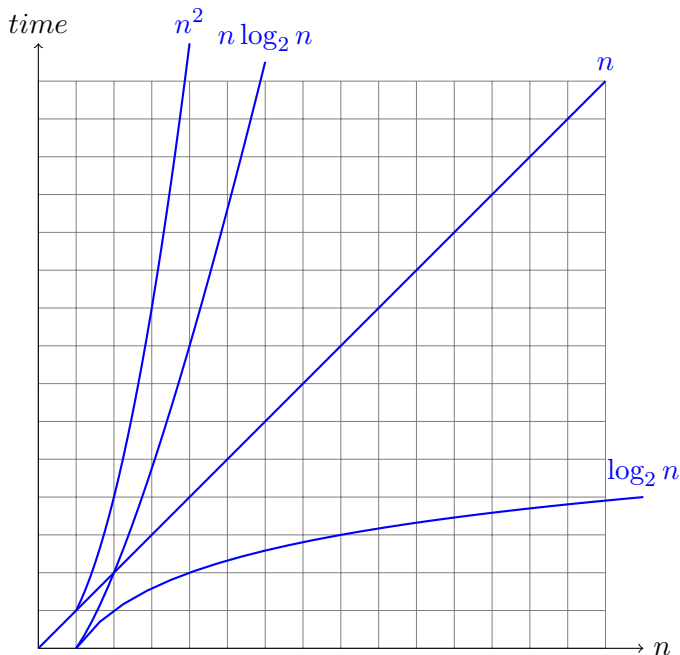


Efficiency	Big-O	Iterations	Est. Time
logarithmic	$O(\log_2 n)$	14	microseconds
linear	$O(n)$	10 000	0.1 seconds
linear log	$O(n \log_2 n)$	140 000	2 seconds
quadratic	$O(n^2)$	10000^2	15-20 min.
polynomial	$O(n^k)$	10000^k	hours
exponential	$O(2^n)$	2^{10000}	intractable
factorial	$O(n!)$	$10000!$	intractable

Assume instruction speed of 1 microsecond and 10 instructions in loop.

$$n = 10000$$

Standard Measures of Efficiency



Complexity of Algorithms

Dr. Nguyen Ho Man Rang



Algorithm Efficiency

Asymptotic Analysis

Problems and common complexities

P and NP Problems



- big-Omega
 - $f(n)$ is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq c.g(n)$ for $n \geq n_0$
- big-Theta
 - $f(n)$ is $\Theta(g(n))$ if there are constants $c' > 0$ and $c'' > 0$ and an integer constant $n_0 \geq 1$ such that $c'.g(n) \leq f(n) \leq c''.g(n)$ for $n \geq n_0$



- Big-Oh: $f(n)$ is $O(g(n))$ if $f(n)$ is asymptotically less than or equal to $g(n)$
- Big-Omega: $f(n)$ is $\Omega(g(n))$ if $f(n)$ is asymptotically greater than or equal to $g(n)$
- Big-Theta: $f(n)$ is $\Theta(g(n))$ if $f(n)$ is asymptotically equal to $g(n)$



Problems and common complexities



Recurrence Equation (Phương trình hồi quy)

An equation or inequality that describes a **function** in terms of its value on **smaller input**.

1	2	3	5	8	13	21	34	55	89
---	---	---	---	---	----	----	----	----	----

$$T(n) = 1 + T(n/2) \Rightarrow T(n) = O(\log_2 n)$$



- **Best case**: when the number of steps is smallest. $T(n) = O(1)$
- **Worst case**: when the number of steps is largest. $T(n) = O(\log_2 n)$
- **Average case**: in between.
 $T(n) = O(\log_2 n)$

Sequential search

8	5	21	2	1	13	4	34	7	18
---	---	----	---	---	----	---	----	---	----

- Best case: $T(n) = O(1)$
- Worst case: $T(n) = O(n)$
- Average case: $T(n) = \sum_{i=1}^n i \cdot p_i$
 p_i : probability for the target being at $a[i]$
 $p_i = 1/n \Rightarrow T(n) = (\sum_{i=1}^n i) / n =$
 $O(n(n+1)/2n) = O(n)$



Quick sort

19	8	3	15	28	10	22	4	12	83
----	---	---	----	----	----	----	---	----	----

Recurrence Equation

$$T(n) = O(n) + 2T(n/2)$$

- Best case: $T(n) = O(n \log_2 n)$
- Worst case: $T(n) = O(n^2)$
- Average case: $T(n) = O(n \log_2 n)$





Algorithm
Efficiency

Asymptotic
Analysis

Problems and
common
complexities

P and NP
Problems

P and NP Problems



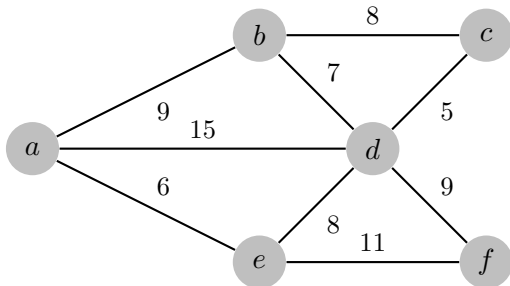
- **P**: Polynomial (can be solved in polynomial time on a **deterministic** machine).
- **NP**: Nondeterministic Polynomial (can be solved in polynomial time on a **nondeterministic** machine).

P and NP Problems

Travelling Salesman Problem:

A salesman has a list of cities, each of which he must visit exactly once. There are direct roads between each pair of cities on the list.

Find the route the salesman should follow for the shortest possible round trip that both starts and finishes at any one of the cities.



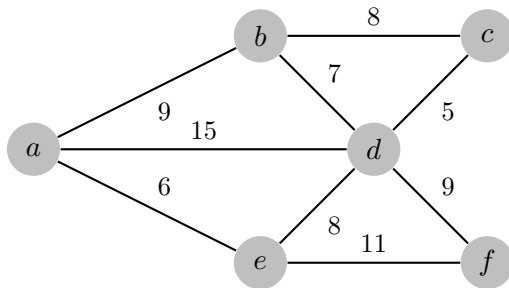
P and NP Problems



Travelling Salesman Problem:

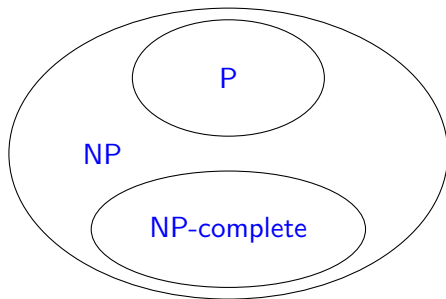
Deterministic machine: $f(n) = n(n-1)(n-2)\dots 1 = O(n!)$

$\Rightarrow$ NP problem



P and NP Problems

NP-complete: NP and every other problem in NP is polynomially reducible to it.



$P = NP?$

